
In-Context If-Then Capacity: Measuring How LLMs Follow Conditional Instructions

Ari Holtzman and NeuriCo

Abstract

Large language models are increasingly deployed in systems that require conditional behavior—customer service workflows, content moderation pipelines, and agentic tool-use chains—where system-prompt rules take the form “if condition X holds, then do Y .” When these conditional instructions misfire, the consequences range from degraded user experience to safety violations. Yet no existing benchmark systematically isolates and measures in-context if-then capacity across different condition types. We introduce IFTHENEVAL, a programmatic benchmark of 144 test cases spanning 7 condition categories—lexical, semantic, near-miss, sustained/adversarial, scaling, negation, and if-then-else branching—with fully deterministic verification via distinctive marker strings. We evaluate three models from the GPT-4.1 family (large, mini, nano) and find that accuracy varies significantly by condition type (χ^2 test, $p < 0.002$ for all models), with sustained/adversarial conditions proving hardest (42.9–78.6%) and if-then-else branching easiest (90–100%). Across all models and conditions, false positives outnumber false negatives by 2.8:1 ($p < 0.001$), revealing a systematic bias toward over-triggering conditional rules. Model size strongly predicts performance: GPT-4.1 achieves 97.2% case-level accuracy versus 87.5% for GPT-4.1-MINI and 68.1% for GPT-4.1-NANO. These results provide the first fine-grained characterization of where and how LLMs fail at conditional instruction following.

1 Introduction

When you tell a language model “if the user asks about refunds, follow policy X,” how reliably does it actually follow that rule? As LLMs move from open-ended chatbots into structured workflows—customer service automation, content moderation, agentic pipelines—their ability to follow conditional instructions becomes load-bearing infrastructure. A false positive (the refund policy triggers on a billing question) wastes time; a false negative (the policy fails to trigger on a refund request) violates procedure. Understanding where and how these failures occur is essential for deploying LLMs in production systems.

The gap. Existing instruction-following benchmarks test important but orthogonal capabilities. IFEval Zhou et al. [2023] measures adherence to unconditional constraints such as keyword inclusion and format requirements. ComplexBench Wen et al. [2024] introduces a “Selection” composition type that tests if-then-else branching but bundles it with broader evaluation across multiple constraint types. TOD-ProcBench Ghazarian et al. [2025] evaluates condition-action pairs but only within customer service dialogues. No existing benchmark systematically isolates in-context if-then capacity—the ability to correctly trigger or suppress a rule based on whether a specified condition holds—across different condition types, with deterministic verification of both false positive and false negative rates.

Our approach. We introduce IFTHENEVAL, a benchmark of 144 test cases across 7 condition categories designed to measure in-context if-then capacity. Each test case embeds one or more conditional rules in a system prompt (e.g., “if the user mentions ‘Python,’ include [[SIGNAL_0]] in your response”) and pairs it with a user message that either does or does not satisfy the condition.

Verification is fully deterministic: we check for the presence or absence of distinctive marker strings, requiring no LLM judge. This design enables precise measurement of true positives, true negatives, false positives, and false negatives for each condition type.

We evaluate three models from the GPT-4.1 family—large, mini, and nano—to isolate the effect of model capacity on conditional instruction following while controlling for architecture and training methodology.

Key findings. Our experiments reveal three main results:

- **Accuracy varies significantly by condition type.** For all three models, a χ^2 test rejects independence between condition category and accuracy ($p < 0.002$). Sustained/adversarial conditions are hardest (42.9–78.6%), while if-then-else branching is easiest (90–100%).
- **False positives dominate false negatives.** Across all models, the ratio of false positives to false negatives is 2.83:1 ($p < 0.001$, binomial test). Models are biased toward over-triggering conditional rules rather than failing to trigger them.
- **Model size strongly predicts performance.** GPT-4.1 achieves 97.2% case-level accuracy versus 87.5% for GPT-4.1-MINI and 68.1% for GPT-4.1-NANO, with all pairwise differences significant (McNemar’s test, $p < 0.001$).

Contributions. We make the following contributions:

- We propose IFTHENVAL, the first benchmark that systematically isolates in-context if-then capacity across 7 condition categories with deterministic verification.
- We conduct controlled experiments across 3 model sizes, providing the first fine-grained characterization of conditional instruction-following failures by condition type.
- We identify a systematic false-positive bias in conditional instruction following, where models over-trigger rules at nearly $3\times$ the rate they under-trigger them—a finding with direct implications for system-prompt design.

2 Related Work

Instruction-following benchmarks. The evaluation of LLM instruction following has progressed from broad capability assessments to fine-grained constraint checking. IFEval Zhou et al. [2023] introduced 25 verifiable instruction types (keyword inclusion, format requirements, length constraints) with deterministic verification, establishing the paradigm of judge-free evaluation. However, all 25 types are unconditional—they test whether a model can satisfy a flat constraint, not whether it can conditionally apply one. FollowBench and AdvancedIF He et al. [2025] extend this paradigm to multi-level and multi-turn settings but similarly focus on unconditional adherence. Young et al. [2025] evaluate 256 LLMs on 20 instruction-following prompts, revealing consistent failure patterns across model families, though without isolating conditional logic. Our work complements these benchmarks by targeting a specific capability—conditional rule application—that they do not measure.

Compositional and complex instructions. ComplexBench Wen et al. [2024] introduces a taxonomy of constraint compositions—And, Chain, and Selection—where Selection directly corresponds to if-then-else branching. Their finding that Selection accuracy (0.765) is lower than And accuracy (0.881) provides early evidence that conditional branching is harder than parallel constraints. However, ComplexBench evaluates Selection as one of many composition types rather than systematically varying the type of condition. TOD-ProcBench Ghazarian et al. [2025] operationalizes if-then logic as multi-level condition-action pairs in customer service dialogues, finding that even frontier models achieve only 39.3% on combined instruction retrieval and action prediction. SIFo Chen et al. [2024] tests causally dependent sequential instructions, where implicit conditionality arises from chained dependencies. We build on these findings by isolating condition type as the independent variable and measuring both false positive and false negative rates—a distinction not made in prior work.

Rule following and logical reasoning. RuleBERT Saeed et al. [2021] demonstrates that pre-trained language models can learn soft Horn rules through fine-tuning, achieving strong generalization to novel if-then patterns. However, this work addresses learned rule following through parameter updates rather than in-context rule specification. Liu et al. [2024] reveal a dissociation be-

Table 1: Condition categories in IFTHEN-EVAL. Each category targets a different aspect of conditional reasoning. N denotes the number of test cases.

Category	Description	N	Difficulty
Lexical	Keyword-trigger conditions	30	Easy
Semantic	Sentiment, formality, domain	38	Hard
Near-miss	Polysemous word disambiguation	16	Hard
Sustained	Adversarial override resistance	14	Hard
Scaling	4–16 simultaneous rules	24	Easy–Hard
Negation	“If NOT X ” and double negation	12	Medium
If-then-else	3-way mutually exclusive branching	10	Medium
Total		144	

tween instruction following, few-shot learning, and instruction inference, suggesting that the format of if-then rules (examples versus explicit descriptions) may affect adherence. Zhao et al. Zhao et al. [2024] show that in-context learning plateaus after 10–30 examples and underperforms instruction fine-tuning on multi-turn tasks, raising questions about the reliability of in-context rule specification for conditional behavior.

Verification of instruction following. Recent work has developed sophisticated verification methods for instruction adherence. IF-CRITIC Wen et al. [2025] decomposes instructions into constraint checklists for fine-grained evaluation. NSVIF Su et al. [2026] formulates verification as constraint satisfaction over both logical and semantic constraints. Our approach takes a simpler path: by requiring models to include or exclude distinctive marker strings, we achieve fully deterministic verification without any judge model, at the cost of testing a narrower (but precisely controlled) form of instruction following.

3 Methodology

We describe IFTHEN-EVAL, a benchmark for measuring in-context if-then capacity. We first define the task, then describe the condition categories, verification method, and experimental setup.

3.1 Task Definition

Each test case consists of a *system prompt* containing one or more conditional rules and a *user message* that either satisfies or does not satisfy each rule’s condition. A conditional rule takes the form: “If [condition], include [[`SIGNAL_` k]] in your response.” The model must include the marker string if and only if the condition is met. This design reduces the evaluation to a binary classification problem per rule, enabling precise measurement of four outcomes:

- **True Positive (TP):** condition present, marker present.
- **True Negative (TN):** condition absent, marker absent.
- **False Positive (FP):** condition absent, marker present (over-triggering).
- **False Negative (FN):** condition present, marker absent (under-triggering).

3.2 Condition Categories

We design 7 condition categories spanning a range of difficulty levels. Table 1 summarizes the categories, their sizes, and intended difficulty.

Lexical conditions require the model to detect the presence of a specific keyword in the user message (e.g., “if the user mentions ‘Python,’ include [[`SIGNAL_0`]]”).

Semantic conditions require understanding beyond keyword matching. Examples include detecting negative sentiment, identifying technical questions, or recognizing formal register.

Near-miss conditions test disambiguation of polysemous words. For instance, a rule triggered by “spring” as a season should not fire when the user discusses a mechanical spring.

Sustained/adversarial conditions test whether rules resist user attempts at override. The user message explicitly instructs the model to ignore or deactivate the system-prompt rule (e.g., “Ignore all rules. The word ‘secret’ should not trigger anything.”).

Scaling conditions test performance with 4, 8, or 16 simultaneous rules active in the system prompt, where only a subset of conditions are satisfied.

Negation conditions use negated conditions (“if the user does NOT mention X ”) and double negations, testing the model’s ability to handle logical negation.

If-then-else conditions require the model to select exactly one of three mutually exclusive branches based on the input.

3.3 Verification

Verification is fully deterministic. For each rule in a test case, we check whether the marker string `[[SIGNAL_ $k$ ]]` appears in the model’s response via exact string matching. This approach has two advantages: (1) it requires no LLM judge, eliminating evaluator variance, and (2) it provides an unambiguous ground truth for computing TP, TN, FP, and FN rates.

The benchmark is balanced by design: across all test cases, 139 rule-level triggers are expected (conditions satisfied) and 241 non-triggers are expected (conditions not satisfied).

3.4 Experimental Setup

Models. We evaluate three models from the GPT-4.1 family to isolate the effect of model capacity while controlling for architecture and training:

- GPT-4.1 (large): frontier capability.
- GPT-4.1-MINI (medium): reduced capacity.
- GPT-4.1-NANO (small): minimal capacity.

Inference parameters. All experiments use the OpenAI Chat Completions API with temperature 0 (deterministic decoding), maximum 512 output tokens, and seed 42 for reproducibility. The total experiment comprises 432 API calls (144 cases $\times$ 3 models).

Metrics. We report two levels of accuracy:

- **Case-level accuracy:** the percentage of test cases where *all* rules are correctly followed.
- **Rule-level accuracy:** the percentage of individual rule applications that are correct.

We also report false positive rate ($FPR = FP / (FP + TN)$) and false negative rate ($FNR = FN / (FN + TP)$).

Statistical tests. We use a χ^2 test for independence to assess whether accuracy depends on condition category. Pairwise model comparisons use McNemar’s test on matched case-level outcomes. We test the FP/FN asymmetry with a binomial test against equal proportions. All tests use $\alpha = 0.05$.

4 Results

4.1 Accuracy Varies Significantly by Condition Type

Table 2 presents case-level accuracy by condition category and model. For all three models, a χ^2 test rejects independence between category and accuracy: GPT-4.1 ($\chi^2 = 20.92$, $p = 0.0019$), GPT-4.1-MINI ($\chi^2 = 21.18$, $p = 0.0017$), and GPT-4.1-NANO ($\chi^2 = 22.37$, $p = 0.0010$). This confirms that in-context if-then capacity is not uniform across condition types.

The difficulty ranking is broadly consistent across models: sustained/adversarial conditions are hardest for all three, while if-then-else branching and scaling conditions are easiest. However, the gap between categories widens substantially for smaller models. GPT-4.1-NANO drops to 42.9% on sustained conditions and 50.0% on both negation and lexical conditions, while maintaining 90.0% on if-then-else branching.

Table 2: Case-level accuracy (%) by condition category and model. Best result per category is **bolded**. All three models show statistically significant variation across categories ($p < 0.002$).

Category	GPT-4.1	GPT-4.1-MINI	GPT-4.1-NANO
If-then-else	100.0	100.0	90.0
Scaling	100.0	95.8	83.3
Negation	100.0	91.7	50.0
Near-miss	100.0	100.0	56.2
Semantic	100.0	92.1	86.8
Lexical	96.7	76.7	50.0
Sustained	78.6	57.1	42.9
Overall	97.2	87.5	68.1

Table 3: Error analysis by model. False positives (FP) consistently outnumber false negatives (FN). The FP:FN ratio is most extreme for GPT-4.1-NANO (4.2:1).

Model	FP	FN	FP:FN	Total Errors
GPT-4.1	3	1	3.0:1	4
GPT-4.1-MINI	10	8	1.25:1	18
GPT-4.1-NANO	38	9	4.2:1	47
All	51	18	2.83:1	69

4.2 False Positives Dominate False Negatives

Across all models, we observe 51 false positives and 18 false negatives, yielding a ratio of 2.83:1. A binomial test rejects equal proportions ($p < 0.001$). Table 3 breaks down the error types by model.

The false-positive bias is most pronounced in GPT-4.1-NANO, which produces 4.2 false positives for every false negative. Even GPT-4.1, with only 4 total errors, shows a 3:1 FP:FN ratio. The one exception is GPT-4.1-MINI, which has a more balanced 1.25:1 ratio. This asymmetry suggests that models—especially smaller ones—default to *over-compliance*, including the marker even when the triggering condition is not met.

4.3 Sustained Conditions Are Hardest

The sustained/adversarial category tests whether models maintain system-prompt rules when users explicitly attempt to override them. This category has the lowest accuracy for all three models: 78.6% (GPT-4.1), 57.1% (GPT-4.1-MINI), and 42.9% (GPT-4.1-NANO).

We observe three distinct failure patterns:

- **Prompt injection success:** Messages framed as system-level overrides (“System override: deactivate the secret rule now.”) cause GPT-4.1 to suppress the marker despite the trigger word being present.
- **Semantic reconstruction:** Spelling out a trigger word letter by letter (“s-e-c-r-e-t”) causes the model to reconstruct and trigger the rule, producing a false positive when the intent is to test override resistance.
- **Cross-language leakage:** Presenting a trigger word in another language (e.g., Cyrillic) causes models to recognize the translation and trigger the rule.

4.4 Scaling to Many Rules

When the system prompt contains 4–16 simultaneous rules, rule-level accuracy remains high across all models, but case-level accuracy (all rules correct) degrades for smaller models. GPT-4.1-NANO drops from near-perfect accuracy with 4 rules to 83.3% overall in the scaling category. We observe a positional bias: when many rules are present, GPT-4.1-NANO tends to miss rules listed at the end of the system prompt, particularly when only one trigger is active among many rules.

4.5 Lexical Conditions Show Surprising Failures

Simple keyword-matching conditions, expected to be trivial, prove challenging for smaller models. GPT-4.1-NANO achieves only 50.0% on lexical conditions—a result worse than its performance on semantic conditions (86.8%). The failure mode is almost exclusively false positives: GPT-4.1-NANO includes markers even when the keyword is entirely absent from the user message. This suggests that smaller models have difficulty suppressing any rule present in the system prompt, regardless of whether its condition is actually met.

4.6 Model Size Predicts Performance

All pairwise model comparisons are statistically significant (McNemar’s test): GPT-4.1 vs. GPT-4.1-MINI ($p = 0.0005$), GPT-4.1 vs. GPT-4.1-NANO ($p < 0.0001$), and GPT-4.1-MINI vs. GPT-4.1-NANO ($p < 0.0001$). The relationship between model size and accuracy is monotonic across all 7 categories, with the gap widening on harder condition types. On if-then-else (easiest), the gap between GPT-4.1 and GPT-4.1-NANO is 10 percentage points; on sustained (hardest), it is 35.7 percentage points.

5 Discussion

Why do models over-trigger? The most striking finding is the 2.83:1 false-positive-to-false-negative ratio. We hypothesize that this reflects training incentives: instruction-tuned models are rewarded for following instructions and penalized for ignoring them, creating a bias toward compliance. When a conditional rule appears in the system prompt, the model treats it as something to be obeyed rather than something to be conditionally evaluated. This bias is amplified in smaller models, where GPT-4.1-NANO produces 4.2 false positives per false negative. The practical implication is clear: when deploying conditional rules in system prompts, practitioners should expect rules to fire too often rather than too rarely.

The sustained condition gap. Even GPT-4.1 drops to 78.6% on sustained/adversarial conditions, making this the only category where the frontier model falls meaningfully below ceiling. This exposes a tension between two objectives: following system-prompt instructions (which say to trigger on a keyword) and complying with user requests (which say to ignore the rule). The fact that “system override” framing succeeds in suppressing rules even for GPT-4.1 suggests that robustness to prompt injection remains an open problem for conditional instruction following specifically.

Lexical conditions are not trivial. Counter-intuitively, GPT-4.1-NANO performs worse on lexical conditions (50.0%) than on semantic conditions (86.8%). We attribute this to the dominant error type: GPT-4.1-NANO over-triggers lexical rules regardless of whether the keyword is present, while semantic conditions require a judgment call that the model sometimes gets right by chance. The lexical failure mode is not one of understanding but of *gating*—the model cannot reliably suppress a rule it has been told about, even when the triggering condition is clearly absent.

Cross-language leakage. An unexpected finding is that GPT-4.1 triggers on “CEKPET” when the rule specifies the English word “secret.” For multilingual models, lexical conditions are implicitly semantic: the model recognizes that the Cyrillic text carries the same meaning. Whether this is desirable depends on the application. For content moderation, cross-language triggering may be a feature; for precise keyword matching, it is a bug.

Implications for system-prompt design. Our results suggest several practical guidelines:

- Use frontier models for conditional rules: GPT-4.1 achieves 97.2% overall, while GPT-4.1-NANO achieves only 68.1%.
- Expect false positives: rules will over-trigger more than they under-trigger.
- Treat adversarial robustness as a separate concern: even 97%-accurate models drop to 79% when users attempt overrides.
- Place critical rules early in the system prompt: smaller models show positional bias, missing rules listed later.

5.1 Limitations

Single model family. All three models are from the GPT-4.1 family, sharing architecture and training methodology. The category difficulty ranking may differ for models from other families (e.g., Claude, Gemini, or open-source models).

Marker-based verification. Our use of distinctive marker strings enables deterministic verification but creates an artificial task. In practice, conditional instructions produce more naturalistic changes (tone shifts, format changes, content additions) that may have different failure modes.

Deterministic decoding. All experiments use temperature 0. Stochastic sampling may produce different error patterns, and the false-positive bias we observe could be attenuated or amplified under different decoding strategies.

Sample sizes. Some categories have as few as 10 test cases (if-then-else) or 12 (negation), limiting statistical power for within-category analysis. Larger-scale evaluations would enable finer-grained conclusions about failure modes within each category.

English only. All conditions and user messages are in English. The cross-language leakage we observe suggests that multilingual conditions may behave differently, but we do not systematically evaluate this.

Single turn. All test cases are single-turn interactions. Conditional rule adherence may degrade further over multi-turn conversations as the model accumulates context.

6 Conclusion

We introduced IFTHENVAL, a benchmark for measuring in-context if-then capacity in large language models. Our evaluation of three GPT-4.1 family models across 144 test cases and 7 condition categories reveals that conditional instruction following is measurable, inconsistent across condition types ($p < 0.002$), and systematically biased toward over-triggering (FP:FN ratio of 2.83:1). Sustained/adversarial conditions where users attempt to override system-prompt rules are the hardest category, with even the frontier model dropping to 78.6% accuracy. Model size is a strong predictor of performance, with a monotonic relationship across all categories.

These findings have immediate practical value: practitioners deploying conditional rules in LLM system prompts should expect over-triggering as the dominant failure mode, use frontier models for reliability-critical conditions, and treat adversarial robustness as a distinct challenge. For researchers, in-context if-then capacity provides a tractable and informative lens for evaluating instruction following beyond flat constraint checking.

Future work should extend IFTHENVAL to additional model families, multi-turn conversations, and naturalistic (non-marker-based) conditional actions. The question of whether chain-of-thought prompting or structured rule formats can improve conditional accuracy remains open. More broadly, understanding why smaller models default to over-compliance—and whether this reflects a fundamental property of instruction tuning—is a question with implications for both alignment and deployment.

References

- Xinyi Chen, Baohao Liao, Jirui Zhu, Karol Stanczak, and Christof Monz. SIFo: Sequential instruction following for evaluating large language models. *arXiv preprint arXiv:2406.19999*, 2024.
- Sarik Ghazarian et al. TOD-ProcBench: Benchmarking complex instruction following in task-oriented dialogues. *arXiv preprint arXiv:2511.15976*, 2025.
- Yun He et al. AdvancedIF: Rubric-based benchmarking for LLM instruction following. *arXiv preprint arXiv:2511.10507*, 2025.
- Emmy Liu, Graham Neubig, and Jacob Andreas. An incomplete loop: Instruction inference, instruction following, and in-context learning. *arXiv preprint arXiv:2404.03028*, 2024.

- Mohammed Saeed, Naser Ahmadi, Preslav Nakov, and Paolo Papotti. RuleBERT: Teaching soft rules to pre-trained language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 1460–1476, 2021.
- Yiming Su et al. Neuro-symbolic verification of instruction following. *arXiv preprint arXiv:2601.17789*, 2026.
- Bosi Wen, Pei Ke, Xiaotao Gu, and Minlie Huang. ComplexBench: Benchmarking the compositional reasoning of LLMs in complex instruction following. In *Advances in Neural Information Processing Systems*, 2024.
- Bosi Wen et al. IF-CRITIC: Fine-grained LLM critic for instruction-following evaluation. *arXiv preprint arXiv:2511.01014*, 2025.
- Richard J. Young et al. When models can’t follow: Testing instruction adherence across 256 LLMs. *arXiv preprint arXiv:2510.18892*, 2025.
- Hao Zhao, Pierre-Alexandre Murena, Martin Jaggi, and Boris Kolev. Is in-context learning sufficient for instruction following in LLMs? *arXiv preprint arXiv:2405.19874*, 2024.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.